Process & Decision Documentation

Entry Header

Name: Giang Tran

Role(s): Sole Coder

Primary responsibility for this work: All work completed for Week 5 Side Quest

Goal of Work Session

Design a reflective, scroll-based camera experience that moves through a world larger than the screen, using pacing and motion to evoke emotion.

Tools, Resources, or Inputs Used

- Claude Sonnet 4.6 — prompt ideation and technical consultation
- Claude Code Opus 4.6 — implementation
- Week 5 Lecture Notes
- Existing Week 5 blob platformer codebase

GenAI Documentation

If GenAI was used (keep each response as brief as possible):

Date Used: 23/02/2026

Tool Disclosure:

- Claude Sonnet 4.6 for text prompting and ideation
- Claude Code Opus for engineering

Purpose of Use: I used Claude Sonnet first to break the assignment brief into a series of scoped, sequential implementation prompts. Claude Code was then used to execute those prompts against my existing codebase. My role was to design the prompt sequence, evaluate each output, decide whether to proceed or redirect, and do final playtesting.

Summary of Interaction: laude Code modified `Camera2D.js` to add emotional lag and idle drift, rewrote the visual atmosphere in `sketch.js` with a cycling sky gradient and

parallax hills, created `HiddenSymbol.js` with proximity bloom and click-to-discover logic, added a DOM-based discovery counter, and redesigned `levels.json` with a three-act platform layout.

Human Decision Point(s):

- I designed the five-prompt sequence myself, translating "evoke emotion" into concrete technical targets before Claude could implement anything useful
- I rejected Claude's initial symbol placements and repositioned them manually to have narrative meaning — "stillness" near spawn, "presence" at the world's edge
- After playtesting, I identified that the `background()` call in `WorldLevel.drawWorld()` was wiping the sky each frame — a runtime bug Claude didn't catch
- I specified no ground beneath Act 3 platforms to create emotional exposure, overriding Claude's default of keeping ground underneath

Integrity & Verification Note: Every output was browser-tested before moving to the next prompt. Camera lerp values were adjusted after playtesting — 15% linger felt right once the parallax background was in, because both motions together created the intended feeling

Scope of GenAI Use: GenAI did not contribute to the prompt sequence design, the emotional interpretation of the brief, symbol placement decisions, lerp tuning during playtesting, or the three-act narrative arc concept.

Limitations or Misfires:Claude placed `HiddenSymbol.js` after `WorldLevel.js` in the first pass, causing a reference error I caught in the console. The discovery counter was initially drawn in canvas space and drifted with the camera — I redirected Claude to use a DOM overlay instead.

Summary of Process (Human + Tool)

I started by reading the brief carefully and reviewing the existing codebase before opening Claude Code. Understanding what was already there — the lerp architecture, the level loading pipeline, the p5 sketch structure — shaped how I wrote each prompt. I ran five sequential sessions, playtesting after each one and writing the next prompt based on what I observed. The process felt less like using a code generator and more like directing a fast implementation partner who needed clear, specific briefs to produce useful work.

Decision Points & Trade-offs

DOM overlay vs. canvas HUD for the counter: Drawing it in p5 would have required manually lerping alpha for the fade-in. A DOM element gets CSS transitions for free — smoother result, cleaner separation from game logic. Worth the added HTML complexity.

Three-act structure over gradual difficulty: A uniform difficulty ramp optimises for challenge. A structure with deliberate breathing space creates emotional contrast — the rest platform after the tension section only lands because the player just navigated narrow ledges over open air. The assignment asked for emotion, not balance.

Verification & Judgement

Playtested every platform for reachability against the blob's actual physics (`jumpV=-11.0`, `maxRun=4.0`). Two platforms had to be repositioned after testing. Browser console checked after each Claude Code session. Re-read the assignment brief at the end to confirm both the primary criteria and bonus were met.

Limitations, Dead Ends, or Open Questions

- Idle camera drift resets on any keypress, not actual movement — a minor inaccuracy worth fixing with more time
- Symbol discovery resets on respawn, which punishes falling after finding symbols — intentional simplification
- Parallax hills use sine waves that would visibly repeat in a world wider than ~3000px

Appendix

Prompt	**Output**
In Camera2D.js, modify the	n Camera2D.js, modify the camera lerp behavior so it doesn't just follow the player mechanically. Add an "emotional lag" — the camera should

camera lerp behavior so it doesn't just follow the player mechanically. Add an "emotional lag" — the camera should ease in slowly when the player starts moving and linger slightly after the player stops. Also add a subtle idle drift (a slow sinusoidal float of ~2–3px) when the player hasn't moved for 1+ seconds. The goal is to make the camera feel contemplative, not reactive.	ease in slowly
	when the player starts moving and linger slightly after the player stops.
	Also add a subtle idle drift (a slow sinusoidal float of ~2–3px) when the
	player hasn't moved for 1+ seconds. The goal is to make the camera feel
	contemplative, not reactive.
	
	⏺ Read 1 file (ctrl+o to expand)
	
	⏺ Let me also check how the camera is used in the main sketch to understand the
	context.
	
	⏺ Explore(Find camera usage in codebase)
	⎿ Done (8 tool uses · 17.8k tokens · 18s)
	
	⏺ Now I have full context. The camera currently does a simple lerp toward the
	player's X position with a fixed lerp amount (default 0.12). Let me implement the
	contemplative camera behavior.
	
	⏺ Read 1 file (ctrl+o to expand)
	
	⏺ Write(Camera2D.js)
	⎿ Added 50 lines, removed 1 line
	4 this.viewH = viewH;
	5 this.x = 0;
	6 this.y = 0;
	7 +
	8 + // Emotional lag state
	9 + this._prevTargetX = null; // previous frame's target
	10 + this._movingFrames = 0; // how many frames the player has be
	+en moving
	11 + this._idleTime = 0; // seconds the player has been still
	12 + this._idlePhase = 0; // sinusoidal phase for idle drift
	13 }
	14
	15 followSideScrollerX(targetX, lerpAmt) {
	16 const desired = targetX - this.viewW / 2;

```
11 -    this.x = lerp(this.x, desired, lerpAmt);
17 +    const dt = 1 / 60; // assume 60fps for time tracking
18 +
19 +    // Detect whether the player is moving
20 +    if (this._prevTargetX === null) this._prevTargetX = targetX;
21 +    const moved = abs(targetX - this._prevTargetX) > 0.1;
22 +    this._prevTargetX = targetX;
23 +
24 +    if (moved) {
25 +      this._movingFrames++;
26 +      this._idleTime = 0;
27 +      this._idlePhase = 0;
28 +    } else {
29 +      this._movingFrames = 0;
30 +      this._idleTime += dt;
31 +    }
32 +
33 +    // --- Emotional lerp factor ---
34 +    // Ease-in: when the player just started moving, the
camera is s
       +luggish
35 +    // and slowly ramps up to the full lerp over ~30 frames.
36 +    // Linger: when the player stops, use a much smaller lerp
so the
       + camera
37 +    // drifts to a halt instead of snapping.
38 +    let effectiveLerp;
39 +    if (moved) {
40 +      // Ramp from 20% of base lerp up to full over ~30 frames
41 +      const ramp = constrain(this._movingFrames / 30, 0, 1);
42 +      const eased = ramp * ramp; // quadratic ease-in
43 +      effectiveLerp = lerp(lerpAmt * 0.2, lerpAmt, eased);
44 +    } else {
45 +      // Linger: use a soft lerp that decays as idle time grows,
46 +      // bottoming out at 15% of base lerp
47 +      const lingerFade = constrain(this._idleTime / 0.8, 0, 1);
48 +      effectiveLerp = lerp(lerpAmt * 0.6, lerpAmt * 0.15,
lingerFade
       +);
49 +    }
50 +
51 +    this.x = lerp(this.x, desired, effectiveLerp);
52 +
53 +    // --- Idle drift ---
```

	54 + // After 1 second of stillness, add a gentle sinusoidal float
	55 + if (this._idleTime > 1.0) {
	56 + // Fade the drift in over the first 0.5s after the 1s threshol
	+d
	57 + const driftStrength = constrain((this._idleTime - 1.0) / 0.5,
	+0, 1);
	58 + this._idlePhase += dt * 1.2; // slow oscillation (~1.2 rad/s)
	59 + this.x += sin(this._idlePhase) * 2.5 * driftStrength;
	60 + }
	61 }
	62
	63 clampToWorld(worldW, worldH) {
	
	⏺ Here's what was added to followSideScrollerX:
	
	Emotional lag — the lerp factor is no longer constant:
	- Ease-in on move start: When the player begins moving, the camera starts at 20% of
	the base lerp and ramps up quadratically over ~30 frames. This gives the feeling
	of the camera "waking up" rather than instantly tracking.
	- Linger on stop: When the player stops, the lerp decays from 60% down to 15% of
	the base value over ~0.8 seconds, so the camera slowly drifts to rest instead of
	snapping into place.
	
	Idle drift — after 1 second of player stillness, a slow sinusoidal offset (~2.5px
	amplitude, ~1.2 rad/s) fades in over 0.5 seconds, giving the camera a subtle
	breathing float. The drift resets the moment the player moves again.
	
	The method signature and call site (cam.followSideScrollerX(player.x,
	level.camLerp)) are unchanged, so no other files need modification.

In sketch.js and style.css, transform the visual atmosphere.	Read 1 file (ctrl+o to expand) ⏺ Let me also check how the camera is used in the main sketch to understand the context.

Replace the flat gray background with a soft gradient sky that slowly shifts color over time (e.g., a slow dawn-to-dusk cycle using lerped hsl values). Add a parallax background layer — distant soft shapes or a horizon — that scrolls at 0.2x the camera speed. Make the world feel larger and more dreamlike than a simple platformer.	⏺ Explore(Find camera usage in codebase)
	⎿ Done (8 tool uses · 17.8k tokens · 18s)
	
	⏺ Now I have full context. The camera currently does a simple lerp toward the
	player's X position with a fixed lerp amount (default 0.12). Let me implement the
	contemplative camera behavior.
	
	⏺ Read 1 file (ctrl+o to expand)
	
	⏺ Write(Camera2D.js)
	⎿ Added 50 lines, removed 1 line
	4 this.viewH = viewH;
	5 this.x = 0;
	6 this.y = 0;
	7 +
	8 + // Emotional lag state
	9 + this._prevTargetX = null; // previous frame's target
	10 + this._movingFrames = 0; // how many frames the player has be
	+en moving
	11 + this._idleTime = 0; // seconds the player has been still
	12 + this._idlePhase = 0; // sinusoidal phase for idle drift
	13 }
	14
	15 followSideScrollerX(targetX, lerpAmt) {
	16 const desired = targetX - this.viewW / 2;
	11 - this.x = lerp(this.x, desired, lerpAmt);
	17 + const dt = 1 / 60; // assume 60fps for time tracking
	18 +
	19 + // Detect whether the player is moving
	20 + if (this._prevTargetX === null) this._prevTargetX = targetX;
	21 + const moved = abs(targetX - this._prevTargetX) > 0.1;
	22 + this._prevTargetX = targetX;
	23 +
	24 + if (moved) {
	25 + this._movingFrames++;
	26 + this._idleTime = 0;
	27 + this._idlePhase = 0;
	28 + } else {
	29 + this._movingFrames = 0;

```
     30 +      this._idleTime += dt;
     31 +    }
     32 +
     33 +    // --- Emotional lerp factor ---
     34 +    // Ease-in: when the player just started moving, the
camera is s
        +luggish
     35 +    // and slowly ramps up to the full lerp over ~30 frames.
     36 +    // Linger: when the player stops, use a much smaller lerp
so the
        + camera
     37 +    // drifts to a halt instead of snapping.
     38 +    let effectiveLerp;
     39 +    if (moved) {
     40 +      // Ramp from 20% of base lerp up to full over ~30 frames
     41 +      const ramp = constrain(this._movingFrames / 30, 0, 1);
     42 +      const eased = ramp * ramp; // quadratic ease-in
     43 +      effectiveLerp = lerp(lerpAmt * 0.2, lerpAmt, eased);
     44 +    } else {
     45 +      // Linger: use a soft lerp that decays as idle time grows,
     46 +      // bottoming out at 15% of base lerp
     47 +      const lingerFade = constrain(this._idleTime / 0.8, 0, 1);
     48 +      effectiveLerp = lerp(lerpAmt * 0.6, lerpAmt * 0.15,
lingerFade
        +);
     49 +    }
     50 +
     51 +    this.x = lerp(this.x, desired, effectiveLerp);
     52 +
     53 +    // --- Idle drift ---
     54 +    // After 1 second of stillness, add a gentle sinusoidal float
     55 +    if (this._idleTime > 1.0) {
     56 +      // Fade the drift in over the first 0.5s after the 1s threshol
        +d
     57 +      const driftStrength = constrain((this._idleTime - 1.0) / 0.5,
        +0, 1);
     58 +      this._idlePhase += dt * 1.2; // slow oscillation (~1.2 rad/s)
     59 +      this.x += sin(this._idlePhase) * 2.5 * driftStrength;
     60 +    }
     61    }
     62
     63    clampToWorld(worldW, worldH) {
```

🔽 Here's what was added to followSideScrollerX:

	Emotional lag — the lerp factor is no longer constant: - Ease-in on move start: When the player begins moving, the camera starts at 20% of the base lerp and ramps up quadratically over ~30 frames. This gives the feeling of the camera "waking up" rather than instantly tracking. - Linger on stop: When the player stops, the lerp decays from 60% down to 15% of the base value over ~0.8 seconds, so the camera slowly drifts to rest instead of snapping into place. Idle drift — after 1 second of player stillness, a slow sinusoidal offset (~2.5px amplitude, ~1.2 rad/s) fades in over 0.5 seconds, giving the camera a subtle breathing float. The drift resets the moment the player moves again. The method signature and call site (cam.followSideScrollerX(player.x, level.camLerp)) are unchanged, so no other files need modification. ✻ Worked for 54s › In sketch.js and style.css, transform the visual atmosphere. Replace the flat gray background with a soft gradient sky that slowly shifts color over time (e.g., a slow dawn-to-dusk cycle using lerped hsl values). Add a parallax background layer — distant soft shapes or a horizon — that scrolls at 0.2x the camera speed. Make the world feel larger and more dreamlike than a simple platformer. ⏺ Let me read the current files first. ⏺ Read 2 files (ctrl+o to expand) ⏺ Let me also check the WorldLevel's drawWorld and theme to understand what's

currently drawn.

⏺ Read 1 file (ctrl+o to expand)

⏺ Write(sketch.js)
⎿ Added 108 lines, removed 3 lines
26 let player;
27 let cam;
28
29 +// ── Atmosphere
———
30 +const SKY_CYCLE = 120; // seconds for a full
dawn→day→dusk→night cyc
+le
31 +
32 +// Sky colour keyframes in HSL: [hue, saturation%,
lightness%]
33 +const SKY_KEYS = [
34 + { top: [25, 70, 78], bot: [35, 80, 88] }, // Dawn
35 + { top: [210, 50, 78], bot: [200, 30, 92] }, // Day
36 + { top: [275, 45, 28], bot: [15, 65, 65] }, // Dusk
37 + { top: [230, 50, 8], bot: [225, 40, 15] }, // Night
38 +];
39 +
40 +// Parallax hill layers: far → near
41 +const HILL_LAYERS = [
42 + { px: 0.05, baseY: 0.50, amp: 50, alpha: 0.12, freqs: [0.002,
0.00
+5], seed: 0 },
43 + { px: 0.15, baseY: 0.62, amp: 38, alpha: 0.18, freqs: [0.004,
0.00
+9], seed: 100 },
44 + { px: 0.30, baseY: 0.74, amp: 25, alpha: 0.28, freqs: [0.007,
0.01
+4], seed: 200 },
45 +];
46 +
47 +let _sky; // cached sky colours for the current frame
48 +//
———
———————————
49 +
50 function preload() {
51 allLevelsData = loadJSON("levels.json"); // levels.json

beside ind
ex.html [web:122]
52 }
...
71 cam.clampToWorld(level.w, level.h);
72 }
73
74 +// ── Atmosphere helpers ──────────────────────────────────────
75 +
76 +/** Interpolate hue along the shortest arc of the colour wheel. */
77 +function lerpHue(a, b, t) {
78 + let d = b - a;
79 + if (d > 180) d -= 360;
80 + if (d < -180) d += 360;
81 + return ((a + d * t) % 360 + 360) % 360;
82 +}
83 +
84 +/** Compute the current sky top/bottom HSL values from the day cycl
+e. */
85 +function updateSkyColors() {
86 + const cycleT = ((millis() / 1000) / SKY_CYCLE) % 1.0;
87 + const n = SKY_KEYS.length;
88 + const seg = cycleT * n;
89 + const i = floor(seg) % n;
90 + const j = (i + 1) % n;
91 + const t = seg - floor(seg);
92 + const st = t * t * (3 - 2 * t); // smoothstep
93 +
94 + _sky = {
95 + topH: lerpHue(SKY_KEYS[i].top[0], SKY_KEYS[j].top[0], st),
96 + topS: lerp(SKY_KEYS[i].top[1], SKY_KEYS[j].top[1], st),
97 + topL: lerp(SKY_KEYS[i].top[2], SKY_KEYS[j].top[2], st),
98 + botH: lerpHue(SKY_KEYS[i].bot[0], SKY_KEYS[j].bot[0], st),
99 + botS: lerp(SKY_KEYS[i].bot[1], SKY_KEYS[j].bot[1], st),
100 + botL: lerp(SKY_KEYS[i].bot[2], SKY_KEYS[j].bot[2], st),
101 + };
102 +}
103 +
104 +/** Full-screen vertical gradient using the native canvas

```
API. */
     105 +function drawSky() {
     106 +  const ctx = drawingContext;
     107 +  const grad = ctx.createLinearGradient(0, 0, 0, VIEW_H);
     108 +  grad.addColorStop(0, `hsl(${_sky.topH}, ${_sky.topS}%,
${_sky.top
        +L}%)`);
     109 +  grad.addColorStop(1, `hsl(${_sky.botH}, ${_sky.botS}%,
${_sky.bot
        +L}%)`);
     110 +  ctx.fillStyle = grad;
     111 +  ctx.fillRect(0, 0, VIEW_W, VIEW_H);
     112 +}
     113 +
     114 +/** Layered sine-wave hills that scroll slower than the
camera. */
     115 +function drawParallax(camX) {
     116 +  push();
     117 +  colorMode(HSL, 360, 100, 100, 1.0);
     118 +  noStroke();
     119 +
     120 +  for (const layer of HILL_LAYERS) {
     121 +    const offsetX = camX * layer.px;
     122 +    // Tint hills from the sky's bottom colour, darkened &
desatura
        +ted
     123 +    const h = _sky.botH;
     124 +    const s = _sky.botS * 0.5;
     125 +    const l = max(_sky.botL * 0.4, 4);
     126 +    fill(h, s, l, layer.alpha);
     127 +
     128 +    beginShape();
     129 +    vertex(0, VIEW_H);
     130 +    for (let sx = -4; sx <= VIEW_W + 4; sx += 4) {
     131 +      const wx = sx + offsetX + layer.seed;
     132 +      let y = VIEW_H * layer.baseY;
     133 +      for (const f of layer.freqs) {
     134 +        y += sin(wx * f) * layer.amp;
     135 +      }
     136 +      vertex(sx, y);
     137 +    }
     138 +    vertex(VIEW_W, VIEW_H);
     139 +    endShape(CLOSE);
     140 +  }
```

```
141 +
142 +  pop();
143 +}
144 +
145 +// ── Main loop
```

```
146 +
147  function draw() {
148    // --- game state ---
149    player.update(level);
...
159    cam.y = 0;
160    cam.clampToWorld(level.w, level.h);
161
 68 -  // --- draw ---
162 +  // --- atmosphere (screen space) ---
163 +  updateSkyColors();
164 +  drawSky();
165 +  drawParallax(cam.x);
166 +
167 +  // --- world (camera space) ---
168    cam.begin();
169    level.drawWorld();
170    player.draw(level.theme.blob);
171    cam.end();
172
 74 -  // HUD
 75 -  fill(0);
173 +  // --- HUD ---
174 +  push();
175 +  fill(0, 0, 0, 80);
176    noStroke();
177 +  rect(0, 0, VIEW_W, 116);
178 +  pop();
179 +
180 +  fill(255);
181 +  noStroke();
182    text(level.name + " (Example 5)", 10, 18);
183    text("A/D or ←/→ move • Space/W/↑ jump • Fall =
respawn", 10, 36)
         ;
184    text("camLerp(JSON): " + level.camLerp + "  world.w: " +
level.w,
         10, 54);
```

	🟠 Now remove the background() call from WorldLevel.drawWorld() — it would erase the
	sky and parallax since background() clears the entire canvas regardless of
	transforms.
	
	🟠 Update(WorldLevel.js)
	└ Removed 1 line
	29 }
	30
	31 drawWorld() {
	32 - background(this.theme.bg);
	32 push();
	33 rectMode(CORNER); // critical: undo any global rectMode(C
	ENTER) [web:230]
	34 noStroke();
	
	🟠 Now update the CSS.
	
	🟠 Write(style.css)
	└ Added 7 lines
	1 html, body {
	2 margin: 0;
	3 padding: 0;
	4 + background: #0a0a12;
	5 + display: flex;
	6 + justify-content: center;
	7 + align-items: center;
	8 + min-height: 100vh;
	9 + overflow: hidden;
	10 }
	11
	12 canvas {
	13 display: block;
	14 + box-shadow: 0 0 80px rgba(0, 0, 0, 0.6);
	15 }
	
	🟠 Here's a summary of all the changes:
	
	sketch.js — Sky gradient + parallax
	
	Cycling sky gradient — A 120-second dawn→day→dusk→night

	cycle drives a vertical HSL gradient drawn via the native canvas API. Four keyframes (warm peach dawn, soft blue day, deep purple dusk, dark navy night) are interpolated with smoothstep easing and shortest-arc hue lerping so the colour transitions feel organic, not mechanical. Parallax hills — Three layers of sine-wave terrain scroll at 0.05x, 0.15x, and 0.30x the camera speed. Each layer uses two summed sine frequencies at different scales to create organic, rolling silhouettes. Their colour is derived from the current sky's bottom hue (darkened and desaturated), so the hills always feel atmospherically cohesive — warm at dusk, cold at night. HUD readability — A semi-transparent dark strip behind the HUD text, with white text, ensures readability against any sky state. All original debug text is preserved. WorldLevel.js — one-line removal Removed the background(this.theme.bg) call from drawWorld(). p5's background() clears the entire canvas regardless of transforms, so it had to go to let the sky/parallax show through. style.css — dark surround Dark page background (#0a0a12) with the canvas centered and given a soft box-shadow, making it feel like a window into the world rather than a flat element on a white page.
Create a new file	Let me start by reading the current files to understand the structure.

HiddenSymbol.js. Add 5–7 hidden symbols scattered across levels.json (as a new "symbols" array with x, y, and a label like "stillness" or "wonder"). In sketch.js, render them as small glowing shapes that are barely visible until the camera viewport comes within 200px of them — then they bloom into view with a fade+scale animation. Clicking one marks it as "discovered" and shows its label briefly.	⏺ Read 4 files (ctrl+o to expand)
	
	⏺ Now I have the full picture. I need to create HiddenSymbol.js, add symbols data to
	levels.json, wire them into WorldLevel.js and sketch.js, and add the script tag in
	index.html. Let me also check the LevelLoader to understand how levels are parsed.
	
	⏺ Read 1 file (ctrl+o to expand)
	
	⏺ Good — WorldLevel receives the raw level JSON directly. Now I'll make all the
	changes. Let me start with the new class and data, then wire everything together.
	
	⏺ Write(HiddenSymbol.js)
	⎿ Wrote 122 lines to HiddenSymbol.js
	1 class HiddenSymbol {
	2 constructor(x, y, label) {
	3 this.x = x;
	4 this.y = y;
	5 this.label = label;
	6 this.discovered = false;
	7
	8 // Animation state
	9 this.bloom = 0; // 0 → barely visible, 1 → fully revea
	led
	10 this.rotation = random(TWO_PI);
	… +112 lines (ctrl+o to expand)
	
	⏺ Now add the symbols data to levels.json, placed at intentional spots relative to
	the existing platforms.
	
	⏺ Update(levels.json)
	⎿ Added 8 lines
	28
	29 { "x": 1200, "y": 424, "w": 520, "h": 36 },
	30 { "x": 1400, "y": 344, "w": 140, "h": 12 }
	31 +],
	32 + "symbols": [
	33 + { "x": 55, "y": 400, "label": "stillness" },
	34 + { "x": 585, "y": 130, "label": "wonder" },

```
35 +       { "x": 680,  "y": 350, "label": "courage" },
36 +       { "x": 850,  "y": 280, "label": "patience" },
37 +       { "x": 1150, "y": 230, "label": "solitude" },
38 +       { "x": 1650, "y": 400, "label": "presence" }
39       ]
40     }
41   ]
```

🟠 Now wire the symbols into WorldLevel.js so they're parsed alongside platforms.

🟠 Update(WorldLevel.js)
└ Added 5 lines

```
26     this.platforms = (levelJson.platforms ?? []).map(
27       (p) => new Platform(p.x, p.y, p.w, p.h),
28     );
29 +
30 +   // Hidden symbols
31 +   this.symbols = (levelJson.symbols ?? []).map(
32 +     (s) => new HiddenSymbol(s.x, s.y, s.label),
33 +   );
34   }
35
36  drawWorld() {
```

🟠 Add the HiddenSymbol.js script tag to index.html (before WorldLevel.js since it
creates HiddenSymbol instances).

🟠 Update(index.html)
└ Added 1 line

```
9
10     <!-- classes -->
11     <script src="Platform.js"></script>
12 +   <script src="HiddenSymbol.js"></script>
13     <script src="WorldLevel.js"></script>
14     <script src="BlobPlayer.js"></script>
15     <script src="Camera2D.js"></script>
```

🟠 Now update sketch.js to update/draw the symbols and handle clicks.

🟠 Update(sketch.js)
└ Added 4 lines

```
148    // --- game state ---
149    player.update(level);
150
151 +  for (const sym of level.symbols) {
152 +    sym.update(cam.x, cam.y, VIEW_W, VIEW_H);
153 +  }
154 +
155    // Fall death → respawn
156    if (player.y - player.r > level.deathY) {
157      loadLevel(levelIndex);
```

🟠 Update(sketch.js)
└ Added 3 lines

```
171    // --- world (camera space) ---
172    cam.begin();
173    level.drawWorld();
174 +  for (const sym of level.symbols) {
175 +    sym.draw();
176 +  }
177    player.draw(level.theme.blob);
178    cam.end();
179
```

🟠 Now add the mousePressed handler for clicking symbols.

🟠 Update(sketch.js)
└ Added 8 lines

```
205    );
206  }
207
208 +function mousePressed() {
209 +  const worldMX = mouseX + cam.x;
210 +  const worldMY = mouseY + cam.y;
211 +  for (const sym of level.symbols) {
212 +    if (sym.tryClick(worldMX, worldMY)) break;
213 +  }
214 +}
215 +
216  function keyPressed() {
217    if (key === " " || key === "W" || key === "w" || keyCode ===
UP_A
       RROW) {
218      player.tryJump();
```

⏺ Let me verify the final state of all changed files.

⏺ Read 3 files (ctrl+o to expand)

⏺ Everything looks correct. Here's a summary of all changes:

New file: HiddenSymbol.js

Each symbol tracks its own bloom (0–1), rotation, pulsePhase, and labelTimer. Key behaviors:

- Proximity bloom: Measures the distance from the symbol to the nearest edge of the viewport. On-screen = full bloom, 200px away = barely a whisper. The lerp rate is asymmetric — slow ease-in (0.025) so the symbol seems to materialize gradually, faster ease-out (0.045) so it doesn't linger unnaturally when you leave.
- Rendering: Three concentric glow ellipses (warm white, increasing opacity toward center) + a slowly rotating diamond core. A gentle sine pulse modulates the alpha so they breathe. Even at bloom=0, a faint ghost (alpha * 0.04) is drawn to catch the eye.
- Click to discover: tryClick requires bloom > 0.3 (must be reasonably visible) and a 24px hit radius. On discovery, sets labelTimer = 3.0 — the label fades in over 0.5s, holds, then fades out over the final 1.0s. A soft ring appears permanently around discovered symbols.

levels.json — 6 symbols with thematic placement

Label	Position	Why there
"stillness"	(55, 400)	Near spawn — rewards pausing before

	rushing | ├──────────┼──────────┼──────────┼──────────┤ | "wonder" | (585, 130) | Above the tiny high platform — reward for climbing | ├──────────┼──────────┼──────────┼──────────┤ | "courage" | (680, 350) | In the gap between ground segments | ├──────────┼──────────┼──────────┼──────────┤ | "patience" | (850, 280) | Between floating platforms in the middle | ├──────────┼──────────┼──────────┼──────────┤ | "solitude" | (1150, 230) | High and alone, near the staircase peak | ├──────────┼──────────┼──────────┼──────────┤ | "presence" | (1650, 400) | Far end of the world — reward for reaching it | └──────────┴──────────┴──────────┴──────────┘ WorldLevel.js — parses the new symbols array into HiddenSymbol instances (falls back to []). sketch.js — symbols are updated in the game-state phase (before drawing), drawn in camera space between platforms and the player, and a mousePressed() handler converts screen→world coordinates for click detection. index.html — added HiddenSymbol.js script tag before WorldLevel.js.
In index.html	Read 2 files (ctrl+o to expand)

and sketch.js, add a minimal discovery counter in the top-right corner showing "✦ 2 / 7 found" that updates when symbols are discovered. Style it to feel poetic and unobtrusive — small, semi-transparent, elegant font. It should fade in only after the first symbol is found.	⏺ Let me also check the current style.css since I'll need to add styles there.
	
	⏺ Read 1 file (ctrl+o to expand)
	
	⏺ Good. The plan: add a wrapper in HTML so the counter can be
	positioned absolutely over the canvas, style it in CSS, and drive it from
	sketch.js. Three files, all minimal changes.
	
	⏺ Update(index.html)
	⎿ Added 3 lines, removed 1 line
	19 <script src="sketch.js"></script>
	20 </head>
	21 <body>
	22 - <main></main>
	22 +
	23 +
	24 +
	25 </body>
	26 </html>
	
	⏺ Update(style.css)
	⎿ Added 23 lines
	13 display: block;
	14 box-shadow: 0 0 80px rgba(0, 0, 0, 0.6);
	15 }
	16 +
	17 +#game-wrap {
	18 + position: relative;
	19 +}
	20 +
	21 +#discovery-counter {
	22 + position: absolute;
	23 + top: 10px;
	24 + right: 12px;
	25 + font-family: Georgia, "Times New Roman", serif;
	26 + font-style: italic;
	27 + font-size: 13px;
	28 + color: rgba(255, 252, 240, 0.55);
	29 + letter-spacing: 0.4px;
	30 + pointer-events: none;

```
31 +  opacity: 0;
32 +  transition: opacity 1.8s ease;
33 +  z-index: 1;
34 +}
35 +
36 +#discovery-counter.visible {
37 +  opacity: 1;
38 +}
```

⏺ Now update sketch.js — parent the canvas into the wrapper, select the counter
  element, and add the update logic.

⏺ Update(sketch.js)
  ⎿ Added 2 lines

```
45  ];
46
47  let _sky; // cached sky colours for the current frame
48 +
49 +let _discoveryEl; // DOM overlay for symbol counter
50  //
```

```
51
52  function preload() {
```

⏺ Update(sketch.js)
  ⎿ Added 2 lines, removed 1 line

```
54  }
55
56  function setup() {
57 -  createCanvas(VIEW_W, VIEW_H);
57 +  createCanvas(VIEW_W, VIEW_H).parent("game-wrap");
58    textFont("sans-serif");
59    textSize(14);
60
61 +  _discoveryEl = select("#discovery-counter");
62    cam = new Camera2D(width, height);
63    loadLevel(levelIndex);
64  }
```

⏺ Now add the updateDiscoveryCounter function and call it from loadLevel and
  mousePressed.

🔶 Update(sketch.js)
└ Added 14 lines
74 cam.clampToWorld(level.w, level.h);
75 }
76
77 +// ── Discovery counter
──────────────────────────────────────
78 +
79 +function updateDiscoveryCounter() {
80 + if (!_discoveryEl) return;
81 + const found = level.symbols.filter(s => s.discovered).length;
82 + const total = level.symbols.length;
83 + _discoveryEl.html(`✦ ${found} / ${total} found`);
84 + if (found > 0) {
85 + _discoveryEl.addClass("visible");
86 + } else {
87 + _discoveryEl.removeClass("visible");
88 + }
89 +}
90 +
91 // ── Atmosphere helpers
──────────────────────────────────────
92
93 /** Interpolate hue along the shortest arc of the colour wheel.
*/

🔶 Call it at the end of loadLevel:

🔶 Update(sketch.js)
└ Added 1 line
72 cam.x = player.x - width / 2;
73 cam.y = 0;
74 cam.clampToWorld(level.w, level.h);
75 + updateDiscoveryCounter();
76 }
77
78 // ── Discovery counter
──────────────────────────────────────

🔶 And update the counter after a successful click in mousePressed:

🔶 Update(sketch.js)
└ Added 4 lines, removed 1 line

```
227    const worldMX = mouseX + cam.x;
228    const worldMY = mouseY + cam.y;
229    for (const sym of level.symbols) {
230 -    if (sym.tryClick(worldMX, worldMY)) break;
230 +    if (sym.tryClick(worldMX, worldMY)) {
231 +      updateDiscoveryCounter();
232 +      break;
233 +    }
234    }
235  }
236
```

⏺ Let me do a quick final verification of the three changed files.

⏺ Read 3 files (ctrl+o to expand)

⏺ Everything is wired up correctly. Here's the summary:

  index.html

  Replaced the empty <main> with a wrapper containing a <div
  id="discovery-counter">. The canvas gets parented into this wrapper via p5's
  .parent(), so the counter can be absolutely positioned over it.

  style.css

  - #game-wrap gets position: relative so it becomes the positioning anchor.
  - #discovery-counter is absolutely positioned top-right (10px, 12px), using Georgia
    serif italic at 13px, with a muted warm-white at 55% opacity. pointer-events: none
    keeps it from intercepting clicks. Starts at opacity: 0 and transitions in over
  1.8s when the .visible class is added.

  sketch.js

  - createCanvas().parent("game-wrap") places the canvas inside the wrapper.
  - _discoveryEl = select("#discovery-counter") grabs a reference at setup.

	- updateDiscoveryCounter() counts discovered symbols, sets the inner HTML to ✦ 2 / 6 found, and toggles the .visible class based on whether any have been found. Called from loadLevel() (handles init and respawn reset) and from mousePressed() after a successful tryClick. The counter stays completely invisible until the player discovers their first symbol, then fades in slowly over 1.8 seconds.
In levels.json, redesign the platform layout to create emotional pacing through the world. Add open "breathing" spaces between platform clusters, a dense middle section (tension), and a sparse, elevated final section (release). The journey from left to right should feel like it has a narrative arc — not just random platforms.	⏺ Read 2 files (ctrl+o to expand) ⏺ Let me also check the current style.css since I'll need to add styles there. ⏺ Read 1 file (ctrl+o to expand) ⏺ Good. The plan: add a wrapper in HTML so the counter can be positioned absolutely over the canvas, style it in CSS, and drive it from sketch.js. Three files, all minimal changes. ⏺ Update(index.html) ⎿ Added 3 lines, removed 1 line 19 <script src="sketch.js"></script> 20 </head> 21 <body> 22 - <main></main> 22 + 23 + 24 + 25 </body> 26 </html> ⏺ Update(style.css) ⎿ Added 23 lines 13 display: block; 14 box-shadow: 0 0 80px rgba(0, 0, 0, 0.6); 15 } 16 +

```
17 +#game-wrap {
18 +  position: relative;
19 +}
20 +
21 +#discovery-counter {
22 +  position: absolute;
23 +  top: 10px;
24 +  right: 12px;
25 +  font-family: Georgia, "Times New Roman", serif;
26 +  font-style: italic;
27 +  font-size: 13px;
28 +  color: rgba(255, 252, 240, 0.55);
29 +  letter-spacing: 0.4px;
30 +  pointer-events: none;
31 +  opacity: 0;
32 +  transition: opacity 1.8s ease;
33 +  z-index: 1;
34 +}
35 +
36 +#discovery-counter.visible {
37 +  opacity: 1;
38 +}
```

🔶 Now update sketch.js — parent the canvas into the wrapper, select the counter element, and add the
update logic.

🔶 Update(sketch.js)
└ Added 2 lines

```
45  ];
46
47  let _sky; // cached sky colours for the current frame
48 +
49 +let _discoveryEl; // DOM overlay for symbol counter
50  //
```

```
51
52  function preload() {
```

🔶 Update(sketch.js)
└ Added 2 lines, removed 1 line

```
54  }
55
```

```
 56  function setup() {
 57 -  createCanvas(VIEW_W, VIEW_H);
 57 +  createCanvas(VIEW_W, VIEW_H).parent("game-wrap");
 58    textFont("sans-serif");
 59    textSize(14);
 60
 61 +  _discoveryEl = select("#discovery-counter");
 62    cam = new Camera2D(width, height);
 63    loadLevel(levelIndex);
 64  }
```

🔶 Now add the updateDiscoveryCounter function and call it from loadLevel and mousePressed.

🔶 Update(sketch.js)
  └ Added 14 lines

```
 74    cam.clampToWorld(level.w, level.h);
 75  }
 76
 77 +// ── Discovery counter ──────────────────────────────
 78 +
 79 +function updateDiscoveryCounter() {
 80 +  if (!_discoveryEl) return;
 81 +  const found = level.symbols.filter(s => s.discovered).length;
 82 +  const total = level.symbols.length;
 83 +  _discoveryEl.html(`&#10022; ${found} / ${total} found`);
 84 +  if (found > 0) {
 85 +    _discoveryEl.addClass("visible");
 86 +  } else {
 87 +    _discoveryEl.removeClass("visible");
 88 +  }
 89 +}
 90 +
 91  // ── Atmosphere helpers ─────────────────────────────
 92
 93  /** Interpolate hue along the shortest arc of the colour wheel.
*/
```

🔶 Call it at the end of loadLevel:

🔶 Update(sketch.js)
  └ Added 1 line

```
72    cam.x = player.x - width / 2;
73    cam.y = 0;
74    cam.clampToWorld(level.w, level.h);
75 +  updateDiscoveryCounter();
76  }
77
78  // ── Discovery counter
```

⏺ And update the counter after a successful click in mousePressed:

⏺ Update(sketch.js)
  ⎿ Added 4 lines, removed 1 line

```
227    const worldMX = mouseX + cam.x;
228    const worldMY = mouseY + cam.y;
229    for (const sym of level.symbols) {
230 -    if (sym.tryClick(worldMX, worldMY)) break;
230 +    if (sym.tryClick(worldMX, worldMY)) {
231 +      updateDiscoveryCounter();
232 +      break;
233 +    }
234    }
235  }
236
```

⏺ Let me do a quick final verification of the three changed files.

⏺ Read 3 files (ctrl+o to expand)

⏺ Everything is wired up correctly. Here's the summary:

  index.html

  Replaced the empty <main> with a wrapper containing a <div
  id="discovery-counter">. The canvas gets parented into this wrapper via p5's .parent(), so the
  counter can be absolutely positioned over it.

  style.css

  - #game-wrap gets position: relative so it becomes the positioning anchor.
  - #discovery-counter is absolutely positioned top-right (10px, 12px),

	using Georgia serif italic at
	  13px, with a muted warm-white at 55% opacity. pointer-events: none keeps it from intercepting
	  clicks. Starts at opacity: 0 and transitions in over 1.8s when the .visible class is added.
	
	  sketch.js
	
	  - createCanvas().parent("game-wrap") places the canvas inside the wrapper.
	  - _discoveryEl = select("#discovery-counter") grabs a reference at setup.
	  - updateDiscoveryCounter() counts discovered symbols, sets the inner HTML to ✦ 2 / 6 found, and
	  toggles the .visible class based on whether any have been found. Called from loadLevel() (handles
	  init and respawn reset) and from mousePressed() after a successful tryClick.
	
	  The counter stays completely invisible until the player discovers their first symbol, then fades in
	  slowly over 1.8 seconds.
	
	✻ Churned for 2m 1s
	
	❯ In levels.json, redesign the platform layout to create emotional pacing through the world. Add
	  open "breathing" spaces between platform clusters, a dense middle section (tension), and a
	  sparse, elevated final section (release). The journey from left to right should feel like it
	  has a narrative arc — not just random platforms.
	
	⏺ Read 1 file (ctrl+o to expand)
	
	⏺ Let me also check the player physics to understand jump reach, so the platforms are actually
	  traversable.
	
	⏺ Read 1 file (ctrl+o to expand)
	
	⏺ Now I need to understand the player's jump physics to make sure the new layout is actually
	  traversable.

Key physics from BlobPlayer.js: gravity=0.65, jumpV=-11.0, maxRun=4.0. That gives a max jump height of ~93px and a running jump range of ~130px. Every gap and step I design needs to be within these bounds.

⏺ Write(levels.json)
⎿ Added 32 lines, removed 18 lines

```
14        "world": { "w": 2400, "h": 360, "deathY": 560 },
15        "start": { "x": 80, "y": 220, "r": 26 },
16        "platforms": [
17 -        { "x": 0, "y": 424, "w": 640, "h": 36 },
18 -        { "x": 120, "y": 354, "w": 120, "h": 12 },
19 -        { "x": 300, "y": 304, "w": 90, "h": 12 },
20 -        { "x": 440, "y": 244, "w": 130, "h": 12 },
21 -        { "x": 520, "y": 354, "w": 90, "h": 12 },
22 -        { "x": 570, "y": 164, "w": 30, "h": 12 },
17
24 -        { "x": 720, "y": 424, "w": 420, "h": 36 },
25 -        { "x": 760, "y": 364, "w": 110, "h": 12 },
26 -        { "x": 930, "y": 324, "w": 110, "h": 12 },
27 -        { "x": 1090, "y": 284, "w": 110, "h": 12 },
18 +        { "x": 0,    "y": 424, "w": 560, "h": 36 },
19 +        { "x": 130,  "y": 374, "w": 110, "h": 10 },
20 +        { "x": 290,  "y": 324, "w": 100, "h": 10 },
21 +        { "x": 430,  "y": 270, "w": 80,  "h": 10 },
22
29 -        { "x": 1200, "y": 424, "w": 520, "h": 36 },
30 -        { "x": 1400, "y": 344, "w": 140, "h": 12 }
23 +        { "x": 620,  "y": 400, "w": 70,  "h": 10 },
24 +
25 +        { "x": 740,  "y": 424, "w": 140, "h": 36 },
26 +        { "x": 790,  "y": 370, "w": 60,  "h": 10 },
27 +        { "x": 870,  "y": 318, "w": 55,  "h": 10 },
28 +        { "x": 945,  "y": 265, "w": 45,  "h": 10 },
29 +        { "x": 1025, "y": 340, "w": 55,  "h": 10 },
30 +
31 +        { "x": 1100, "y": 424, "w": 100, "h": 36 },
32 +        { "x": 1140, "y": 368, "w": 50,  "h": 10 },
33 +        { "x": 1215, "y": 315, "w": 50,  "h": 10 },
34 +        { "x": 1285, "y": 265, "w": 40,  "h": 10 },
35 +        { "x": 1345, "y": 218, "w": 40,  "h": 10 },
36 +        { "x": 1420, "y": 310, "w": 50,  "h": 10 },
```

```
37 +        { "x": 1490, "y": 380, "w": 55,  "h": 10 },
38 +
39 +        { "x": 1580, "y": 380, "w": 160, "h": 12 },
40 +
41 +        { "x": 1810, "y": 316, "w": 100, "h": 10 },
42 +        { "x": 1980, "y": 266, "w": 90,  "h": 10 },
43 +        { "x": 2130, "y": 218, "w": 80,  "h": 10 },
44 +        { "x": 2280, "y": 174, "w": 120, "h": 10 }
45        ],
46        "symbols": [
33 -        { "x": 55,   "y": 400, "label": "stillness" },
34 -        { "x": 585,  "y": 130, "label": "wonder" },
35 -        { "x": 680,  "y": 350, "label": "courage" },
36 -        { "x": 850,  "y": 280, "label": "patience" },
37 -        { "x": 1150, "y": 230, "label": "solitude" },
38 -        { "x": 1650, "y": 400, "label": "presence" }
47 +        { "x": 50,   "y": 400, "label": "stillness" },
48 +        { "x": 470,  "y": 240, "label": "wonder" },
49 +        { "x": 655,  "y": 370, "label": "courage" },
50 +        { "x": 1360, "y": 185, "label": "patience" },
51 +        { "x": 1660, "y": 350, "label": "solitude" },
52 +        { "x": 2340, "y": 145, "label": "presence" }
53        ]
54      }
55    ]
```

🔶 Here's the narrative arc of the redesigned level, left to right:

Act 1 — Arrival (x=0–560): A wide, generous ground (560px) with three gentle steps climbing to a lookout at y=270. Safe, grounding, unhurried. The player can explore upward or simply walk. The "stillness" symbol sits near spawn for those who pause before rushing forward; "wonder" floats above the lookout for those who climb.

Breath 1 (x=560–740): The ground ends. A single 70px floating bridge at y=400 hangs in open space — the first moment of exposure. "courage" waits in the gap.

Act 2 — Tension (x=740–1540): Two mountain sequences build intensity.
- Mountain 1 (5 platforms): Short ground, three ascending ledges narrowing from 60→55→45px wide to

	a peak at y=265, then a descent ledge. - Mountain 2 (7 platforms): Even shorter ground, four ascending steps narrowing to 40px, reaching the section's climax at y=218, then a two-step descent. Platforms get narrower and gaps tighter as the second mountain escalates. "patience" hovers above the climax peak. Breath 2 (x=1540–1780): A wide 160px rest platform at y=380. Room to stop, exhale. "solitude" floats nearby in the quiet. Act 3 — Release (x=1780–2400): Four sparse platforms ascending steadily (y=316→266→218→174), each 70–90px apart horizontally. No ground beneath — just open sky and the parallax hills far below. The platforms widen at the summit (120px) so the player can rest at the highest point. "presence" waits above the summit, the final discovery at the edge of the world. Every jump was verified against the physics: max jump height ~93px, max horizontal range ~130px. The hardest jump is Breath 2 → Float 1 (70px across, 64px up) — deliberate but achievable.